

NAME

Shourya Thakur

INDEX

STD

DSD

DIV.

ROLL NO.

24BCI0171

S.No.	Date	Topic	Page No.	Signature
1		Write a Verilog model and Verify truth table		Kul
2		Write verilog code for $F(A, B, C) = \bar{A} + B\bar{C} + \bar{A}B\bar{C}$, verify using gate level & data flow		Kul
3		Write verilog code for verifying half adder & half subtractor		gobin
4		Verilog code for Full Adder & Full Subtractor		
5		Verilog code for 2:4 decoder		Dalit 27-09-2021
6	28/09/2021	Verilog Code for 8:1 Mux & 1:8 demux		
7	4/9/2021	4 bit ripple adder		O/N gaur A Sh
8		3 bit Comparator		Dalit 12-09-2021
9	11/9/2021	2 bit by 2 bit Multiplier		Sury 11/9/21
10		Flip-flop all ↳ (SR, JK, D, T)		
11		SISO & SIPO (Shift Register)		Dalit 16-09-2021
12		Counter - 3 (Up down)		

(MOON, Johnson)

Assessment - 5

Aim :- To design, simulate, and verify the working of SR, JK, D, and T flip-flops using ModelSim.

Tools required :- ModelSim

Theory :-

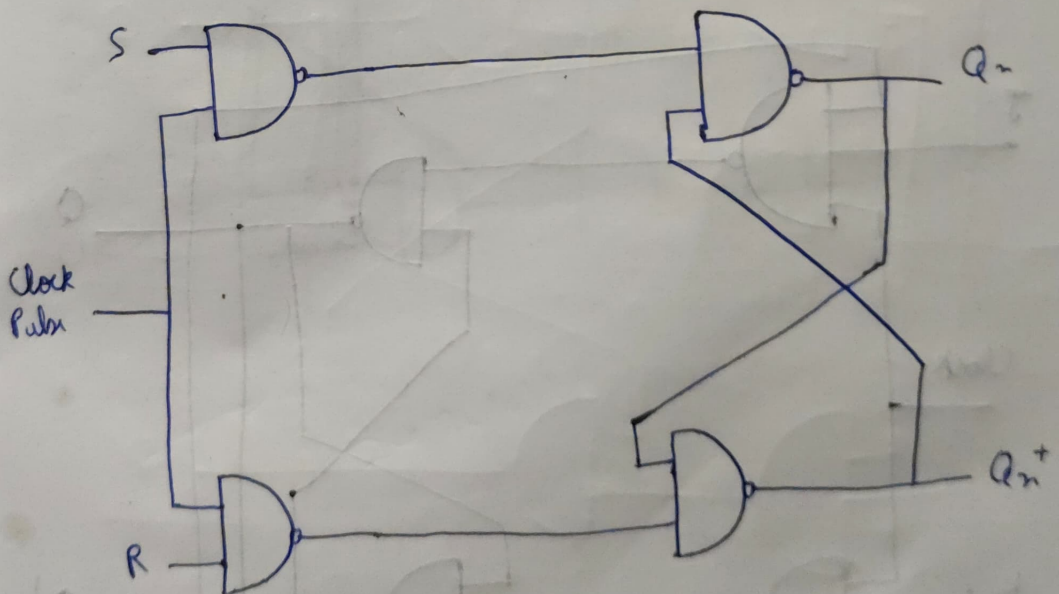
Flip-flops are bistable memory elements used for storing one bit of data. They change state on the application of a clock pulse and are the fundamental building blocks of sequential logic circuits. The SR flip-flop has set and reset inputs used to store binary data, though it suffers from an undefined state when both inputs are 1. The JK flip-flop overcomes this problem and toggles its output when both inputs are high. The D flip-flop, also known as the data or delay flip-flop, transfers the input value to the output on every clock edge, making it ideal for data storage and synchronization. The T flip-flop toggles its state when the input T is high and retains its state when T is low, making it useful in counters and frequency dividers.

1) SR - Flip Flop

↳ Truth Table

S	R	Q (next)
0	0	Q (previous)
0	1	0
1	0	1
1	1	Undefined

↳ Diagram



sn flip flop

```
module sn - ff (clk, rst, s, n, z);  
input clk, rst, s, n;  
output reg z;  
always @ (posedge clk or posedge rst)
```

```
begin
```

```
if (rst == 1)
```

```
z <= 0;
```

```
else if (s == 0 && n == 0)
```

```
z <= ~z;
```

```
else if (s == 0 && n == 1)
```

```
z <= 0;
```

```
else if (s == 1 && n == 0)
```

```
z <= 1;
```

```
else if (s == 1 && n == 1)
```

```
z <= 1'b x;
```

```
end
```

```
endmodule
```

```
module tb();
```

```
reg clk, rst, s, n;
```

```
wire z;
```

```
sn - ff uut (clk, rst, s, n, z);
```

```
always begin
```

```
#10 clk = ~clk
```

```
end
```

```
initial
```

```
begin
```

```
clk = 0; rst = 0; s = 0; n = 0;
```

```
#5 rst = 1;
```

```

# 5 A1 = 0; A = 1; A = 0;
# 20 S = 0; A = 1;
# 20 S = 1; A = 1;
# 20 S = 0; A = 0;
# 20 A1 = 1;
# 5 A1 = 0;
$ top;

```

end
end no dub

fall
0/8 28 24 B 20 17 10

Clock	S	R	Q	Q'
0	x	x	Q.	Q.
↑	0	1	0	1
↑	0	0	0	1
↑	1	0	1	0
↑	0	0	1	0
↑	1	1	0	0

Comment

No change

Q = 0 (new)

No change

Q = 1 (new)

No change

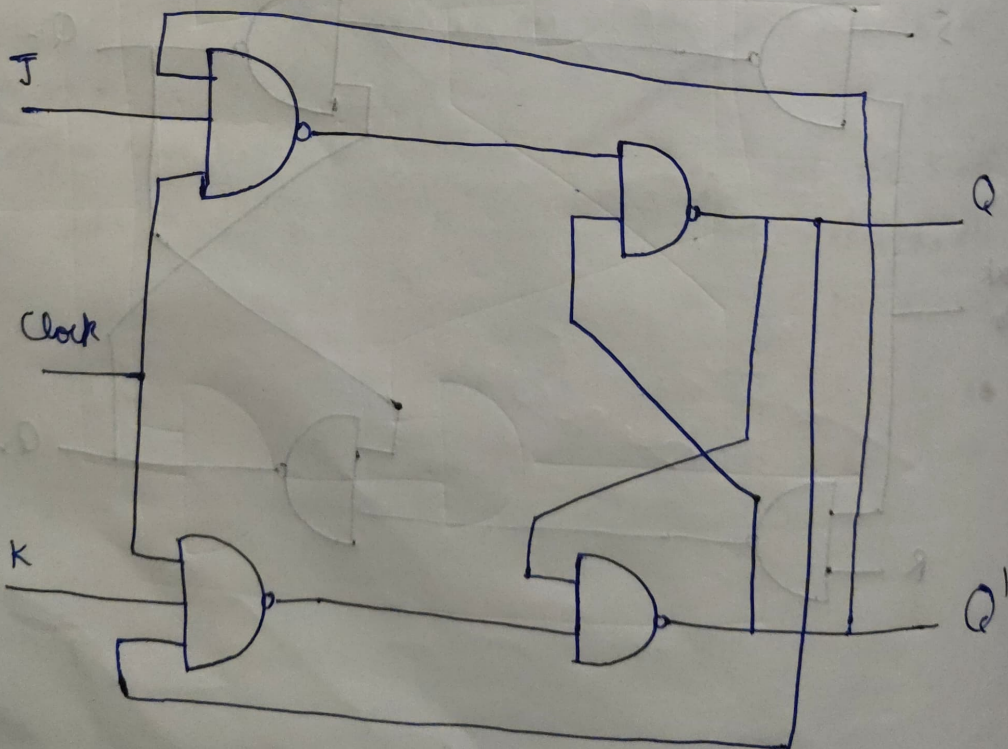
forbidden

2) JK Flip Flop

↳ Truth Table

J	K	$Q(\text{next})$
0	0	$Q(\text{previous})$
0	1	0
1	0	1
1	1	Q'

↳ diagram



JK flip flop

module JK - ff (clk, rst, j, k, q);

input clk, rst, j, k;

output reg q;

always @ (posedge clk or posedge rst)

begin

if (rst == 1)

q <= 0;

else if (j == 0 && k == 0)

q <= q;

else if (j == 0 && k == 1)

q <= 0;

else if (j == 1 && k == 0)

q <= 1;

else if (j == 1 && k == 1)

q <= ~q;

end

end module

module tb1();

reg clk, rst, j, k;

wire q;

JK - ff uut (clk, rst, j, k, q);

always begin

#10 clk = ~clk;

end

initial

begin

clk = 0; rst = 0; j = 0; k = 0;

#5 rst = 1;

#5 rst = 0; j = 0; k = 0;

```

#25 j = 0 ; k = 1;
#25 j = 1 ; k = 1;
#25 j = 1 ; k = 0;
#5 nst = 1;
#25 nst = 0 ; j = 1 ; k = 1;
$ stop;
end
end module

```

clock	j	k	Q	Q'	Comment
0	x	x	Q ₀	Q ₀	N. change
↑	0	1	0	1	Q = 0
↑	0	0	0	1	N. change
↑	1	0	1	0	Q = 1
↑	0	0	1	0	N. change
↑	1	1	0	1	toggle

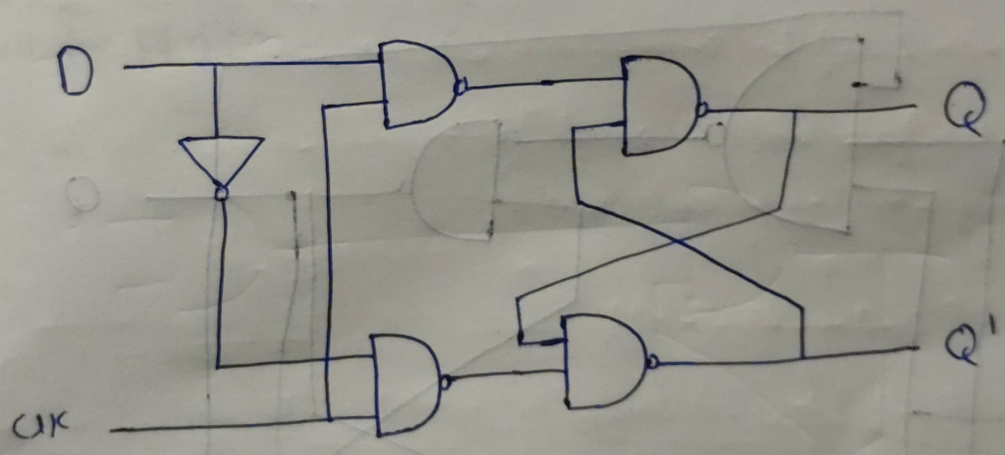
01620000
 25161000
 11020000

3) D Flip Flop

→ Truth Table

D	Q (next)
0	0
1	1

→ Diagram



D flip flop

```
module d_ff (clk, rst, d, q);  
  input clk, rst, d;  
  output reg q;  
  always @ (posedge clk or posedge rst)
```

```
begin
```

```
  if (rst == 1)
```

```
    q <= 0;  
  else  
    q <= d;  
end  
endmodule
```

Handwritten notes:
010
8.04/10
18
24/02/2017

```
module tb();
```

```
  reg clk, rst, d;
```

```
  wire q;
```

```
  d_ff uut (clk, rst, d, q);
```

```
  always begin
```

```
    #10 clk = ~clk;
```

```
  end
```

```
  initial
```

```
  begin
```

```
    clk = 0; rst = 1; d = 0;
```

```
    #5 rst = 1;
```

```
    #5 rst = 0; d = 0;
```

```
    #15 d = 1;
```

```
    #10 d = 0;
```

```
    #15 d = 1;
```

```
    #5 rst = 1;
```

```
    #5 rst = 0; d = 1;
```

```
  $stop;  
end  
endmodule
```

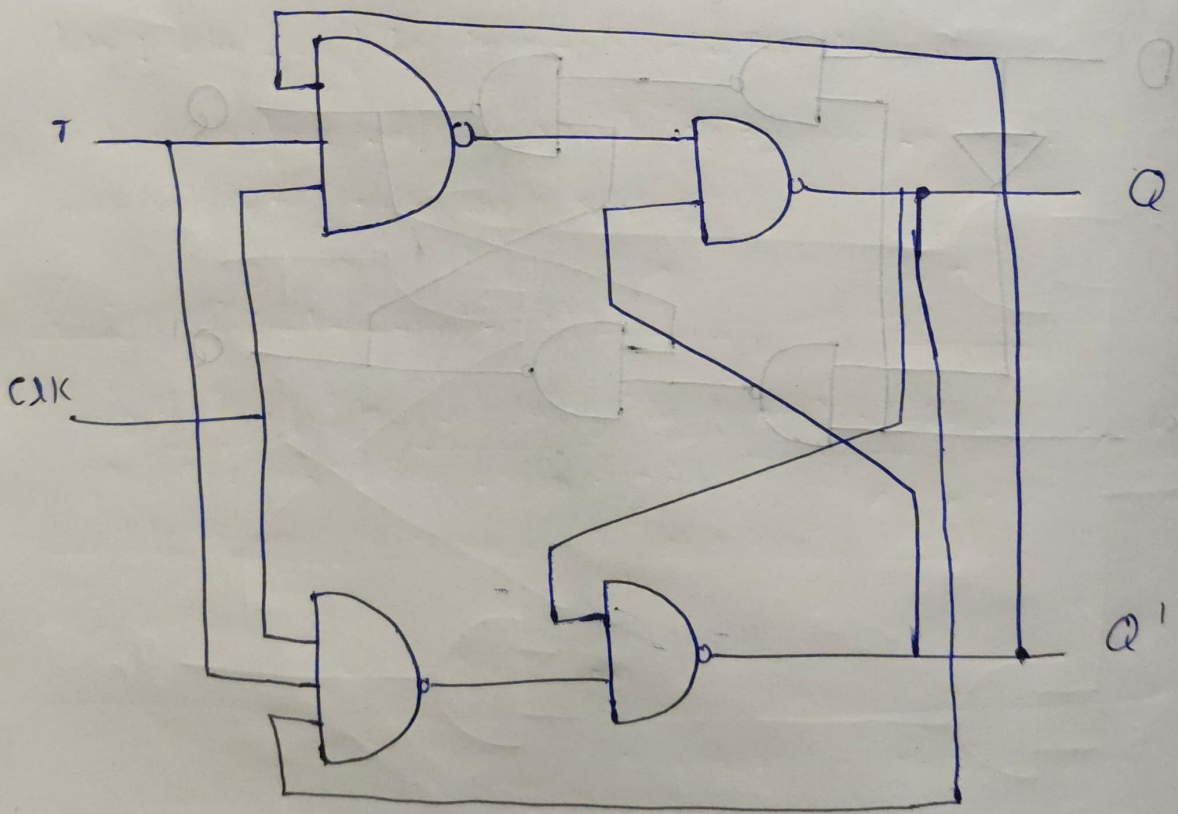

	$Q(n+1)$	State
0	0	read
0	1	set
1		

4) T Flip Flop

↳ Truth Table

T	Q(next)
0	Q (previous)
1	Q'

↳ Diagram



→ flip flop

```
module t_ff (clk, rst, t, q);
```

```
input clk, rst, t;
```

```
output reg q;
```

```
always @ (posedge clk or posedge rst)
```

```
begin
```

```
if (rst == 1)
```

```
q <= 0;
```

```
else if (t == 0)
```

```
q <= q;
```

```
else
```

```
q <= ~q;
```

```
end
```

```
endmodule
```

```
module tb();
```

```
reg clk, rst, t;
```

```
wire q;
```

```
t_ff out (clk, rst, t, q);
```

```
always begin
```

```
#10 clk = ~clk;
```

```
end
```

```
initial
```

```
begin
```

```
clk = 0; rst = 1; t = 0;
```

```
// #5 rst = 1;
```

```
#5 rst = 0; t = 0;
```

```
#15 t = 1;
```

```
#10 t = 0;
```

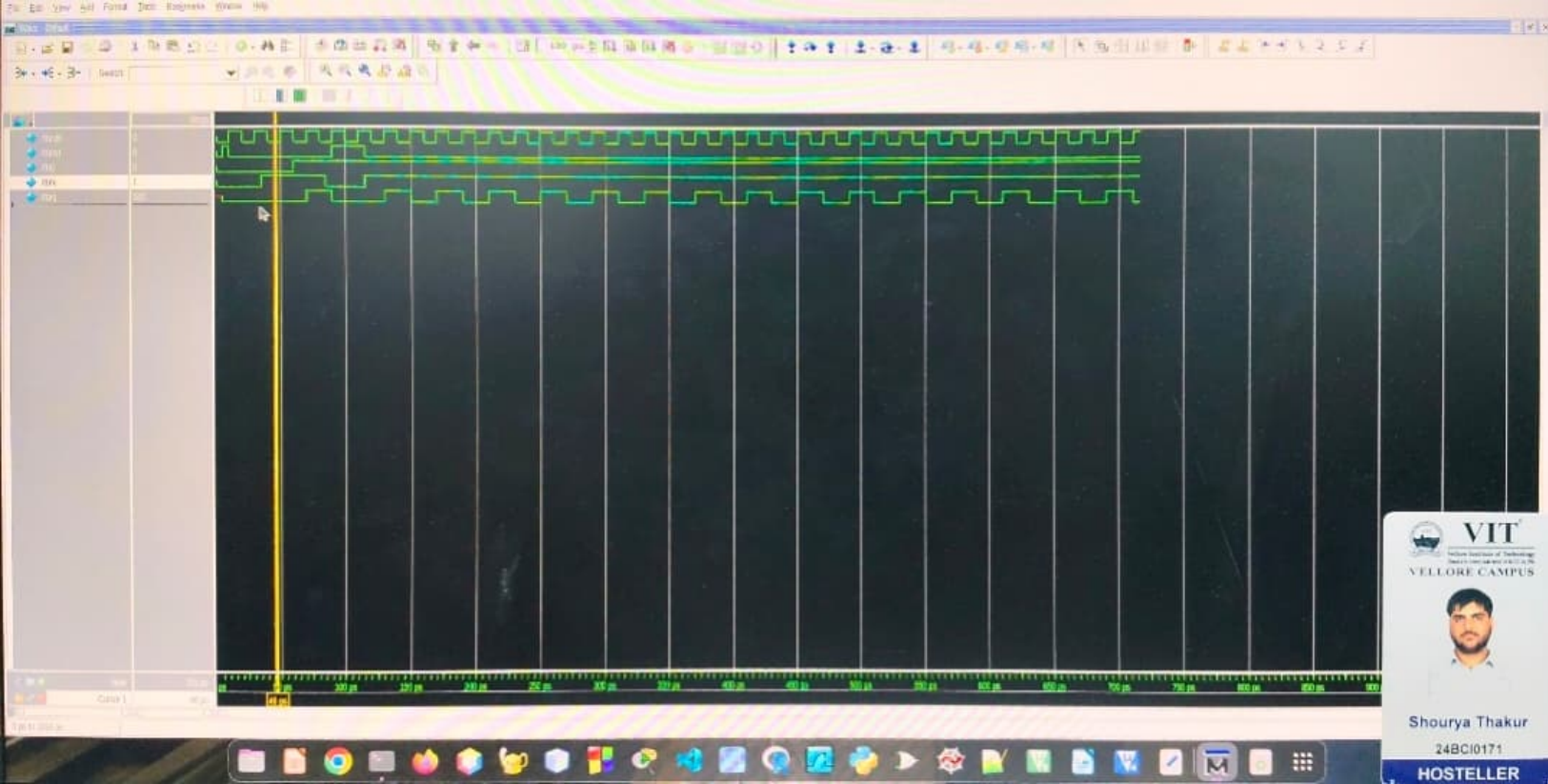
#15 $t = 1;$
 # 5, $1st = 0; t = 1;$
 \$ stop?
 and module

clock	T	Q	Q'	Comment
0	x	Q ₀	Q ₀ '	No change change
↑	0	Q ₀	Q ₀ '	No change
↑	1	Q ₀ '	Q ₀	Toggle

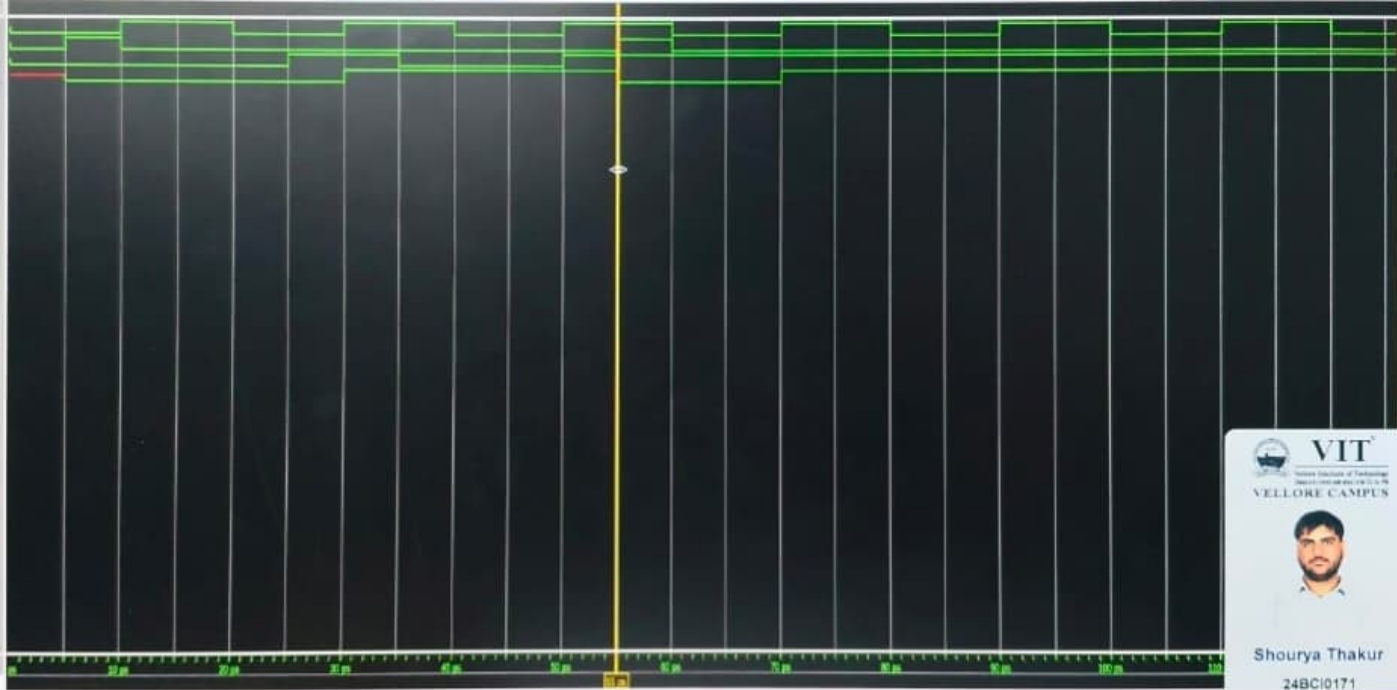
opp verified

Writp

18-09-2025



Tools
Layers
View
Help



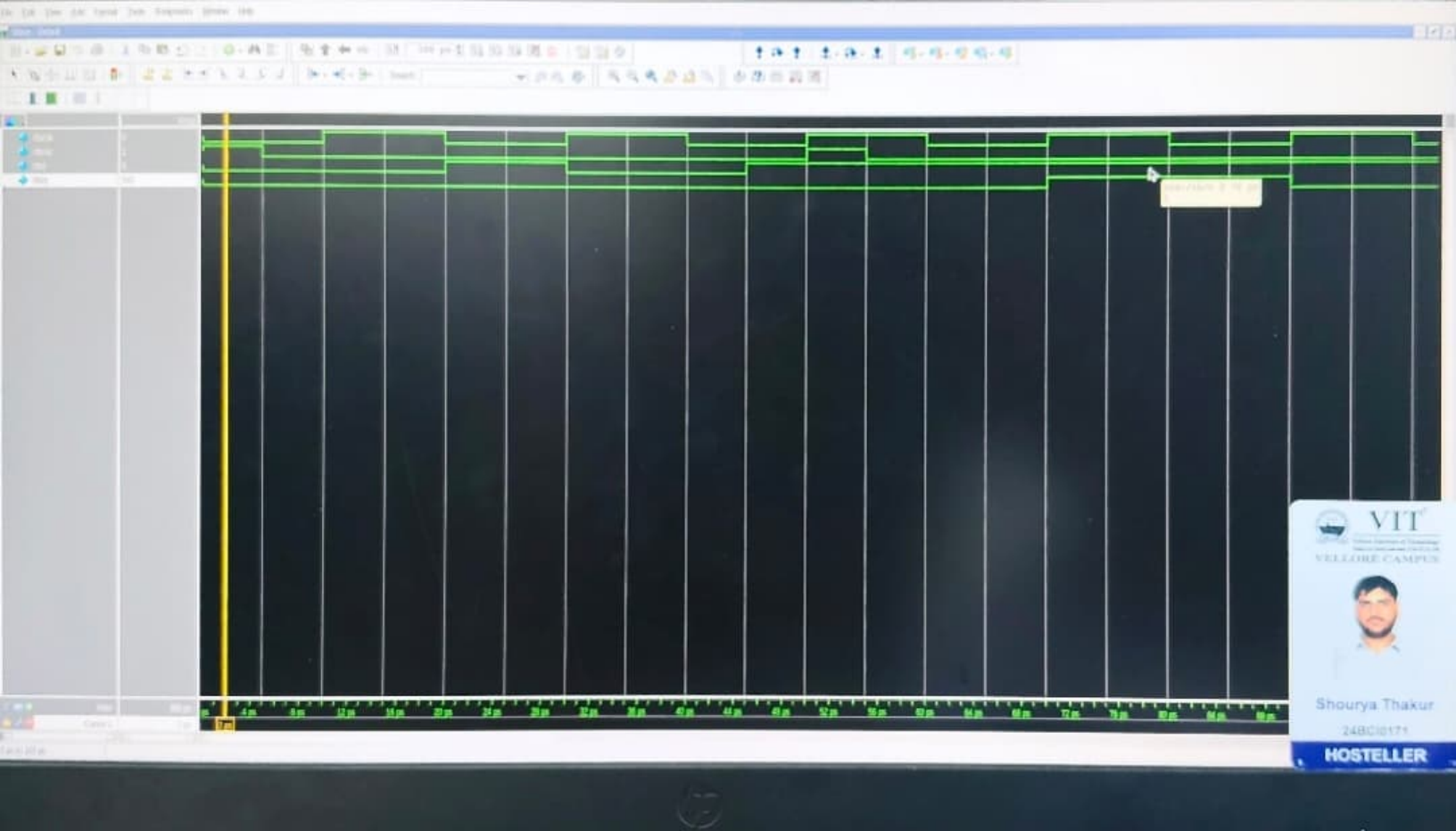
VIT
Vellore Institute of Technology
VELLORE CAMPUS



Shourya Thakur

24BCI0171

HOSTELLER



VIT
Vellore Institute of Technology
VELLORE CAMPUS



Shourya Thakur

24BC0171

HOSTELLER

Inference :-

The SR, JK, D and T flip flop were successfully designed, simulated, and verified using ModelSim. The simulation waveforms matched their respective truth tables and theoretical behavior. It was observed that each flip-flop stores one bit of information and changes state based on its inputs and the clock signal.

↳ The SR flip flop performs basic set and reset operations

↳ The JK flip flop ~~eliminates~~ eliminates the invalid state of SR and provides toggling.

↳ The D flip flop transfers data input to output on every clock pulse.

↳ The T flip flop toggles its state with each clock pulse when $T = 1$.

Thus, the experiment verifies the correct workings and functional differences of all four flip-flops.

Aim :- To design, simulate, and verify the working of Serial-in-serial-out (SISO) and Serial-in-parallel-out (SIPO) shift registers using ModelSim.

Tools :- ModelSim

Theory :-

A shift register is a sequential circuit made up of a chain of flip-flops connected in series, where the output of one flip-flop acts as the input for the next. Shift registers are used for data storage, data transfer, and bit manipulation in digital systems.

Each clock pulse shifts the stored data by one position either left or right, depending on the configuration.

- SISO (Serial-in Serial-out):

In this type, data is fed serially (bit by bit) into the register, and the output is also taken serially from the last flip-flop. It's used when data transmission occurs one bit at a time.

- SIPO (Serial-in Parallel-out):

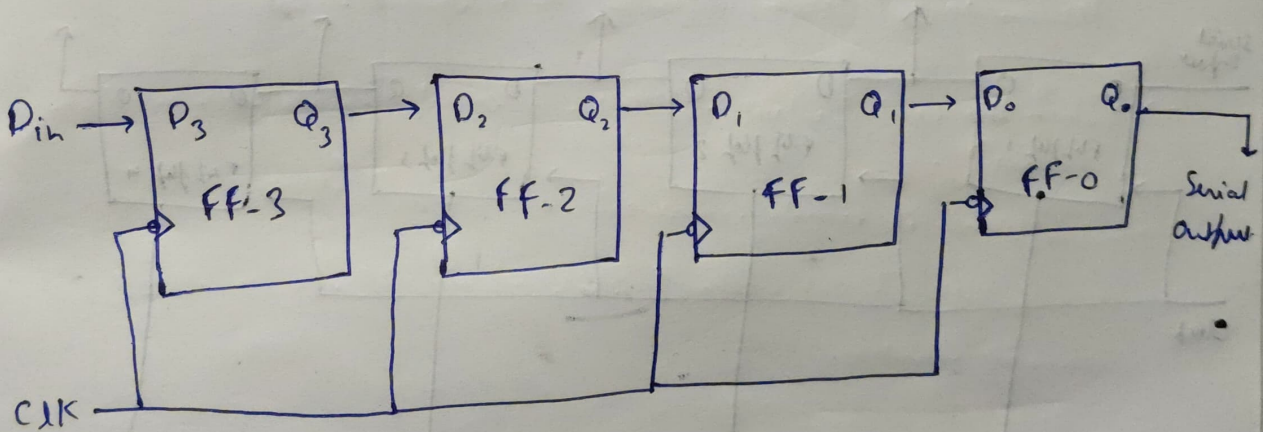
In this type, data is entered serially but is available simultaneously (in parallel) from all flip-flop outputs. It's useful when serial data needs to be converted into parallel form for further processing.

1) SISO (Serial-in Serial-out):

↳ Truth table

clock Pulse	Serial Input	Q_0	Q_1	Q_2	Q_3	Output (Serial out)
1	1	1	0	0	0	0
2	0	0	1	0	0	0
3	1	1	0	1	0	0
4	0	0	1	0	1	1

↳ Diagram



SISO

```
module siso (so, si, clk, rst);
```

```
output so;
```

```
input si, clk, rst;
```

```
reg [2:0] q;
```

```
always @ (posedge clk or posedge rst)
```

```
begin
```

```
if (rst)
```

```
q <= 4'b0;
```

```
else
```

```
q <= {si, q[3:1]};
```

```
end
```

```
assign so = q[0];
```

```
endmodule
```

```
module siso_test;
```

```
width so;
```

```
reg si, clk, rst;
```

```
siso s1 (so, si, clk, rst);
```

```
always
```

```
#5 clk = ~clk;
```

```
initial
```

```
begin
```

```
si = 0, clk = 0;
```

```
#10 rst = 0;
```

```
#10 si = 1;
```

```
#10 si = 0;
```

```
#10 si = 0;
```

```
#10 si = 1;
```

```
#10 si = 1;
```

```
#50 $stop;
```

```
end
```

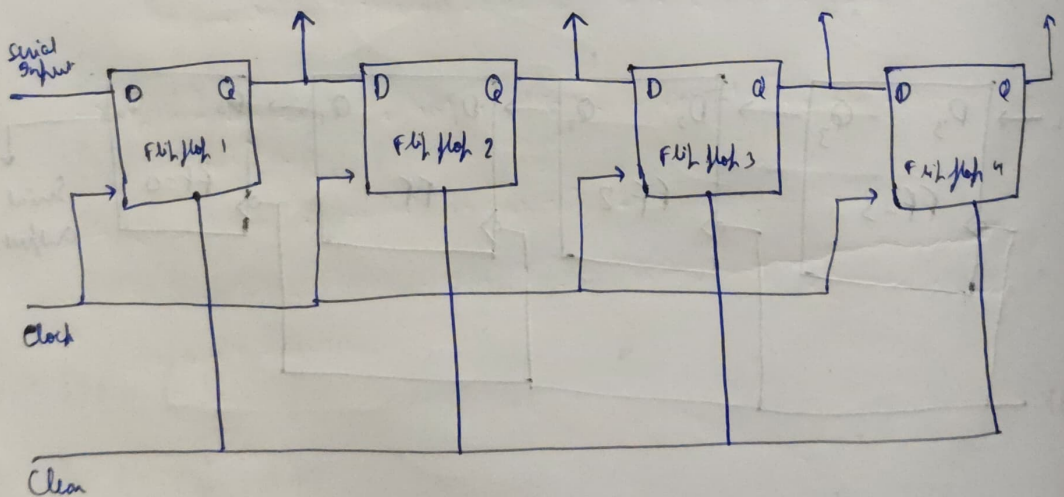
no
1600
24000000

2) SISO (Serial-In Parallel-Out)

↳ Truth Table

Clock Pulse	Serial Input	Q ₀	Q ₁	Q ₂	Q ₃ (Parallel Out)
1	1	1	0	0	0
2	0	0	1	0	0
3	1	1	0	1	0
4	1	1	1	0	1

↳ Diagram



5180

```
module sifo (q, si, clk, rst);  
    output [3:0] q;  
    input si, clk, rst;  
    reg [3:0] q;  
    always @ (posedge clk or posedge rst)
```

```
begin
```

```
    if (rst)
```

```
        q <= 4'b0;
```

```
    else
```

```
        q <= {si, q[3:1]};
```

```
    end
```

```
endmodule
```

Q. Verify the output
for input
24BC20171

```
module sifo-test;
```

```
    wire [3:0] q;
```

```
    reg si, clk, rst;
```

```
    sifo s1 (q, si, clk, rst);
```

```
    always
```

```
        #5 clk = ~clk;
```

```
    initial
```

```
        begin
```

```
            si = 0; clk = 0; rst = 1;
```

```
            #10 rst = 0;
```

```
            #10 si = 1;
```

```
            #50 si = 0;
```

```
            #10 si = 0;
```

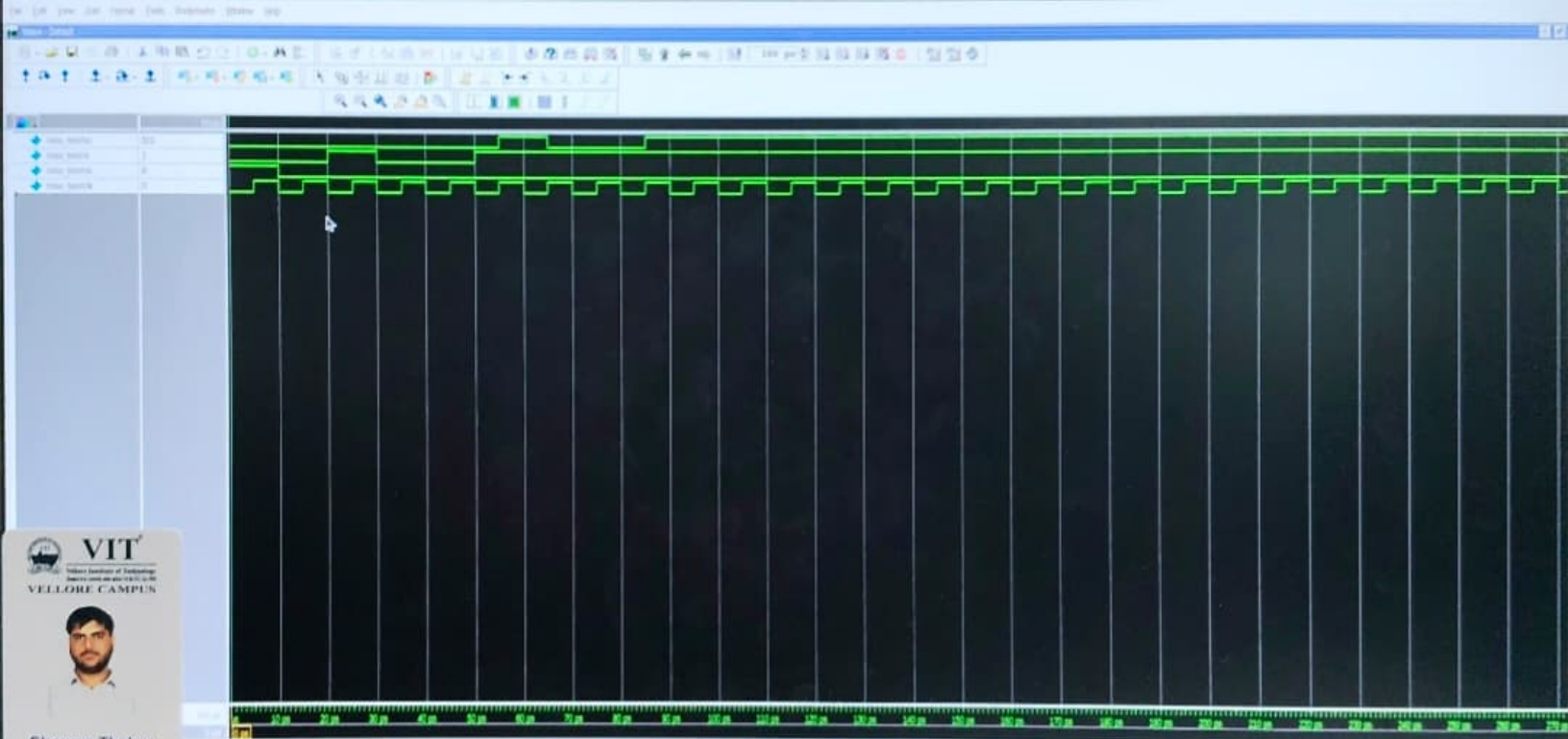
```
            #10 si = 1;
```

```
            #10 si = 1;
```

```
            #50 $stop;
```

```
        end
```

```
endmodule
```



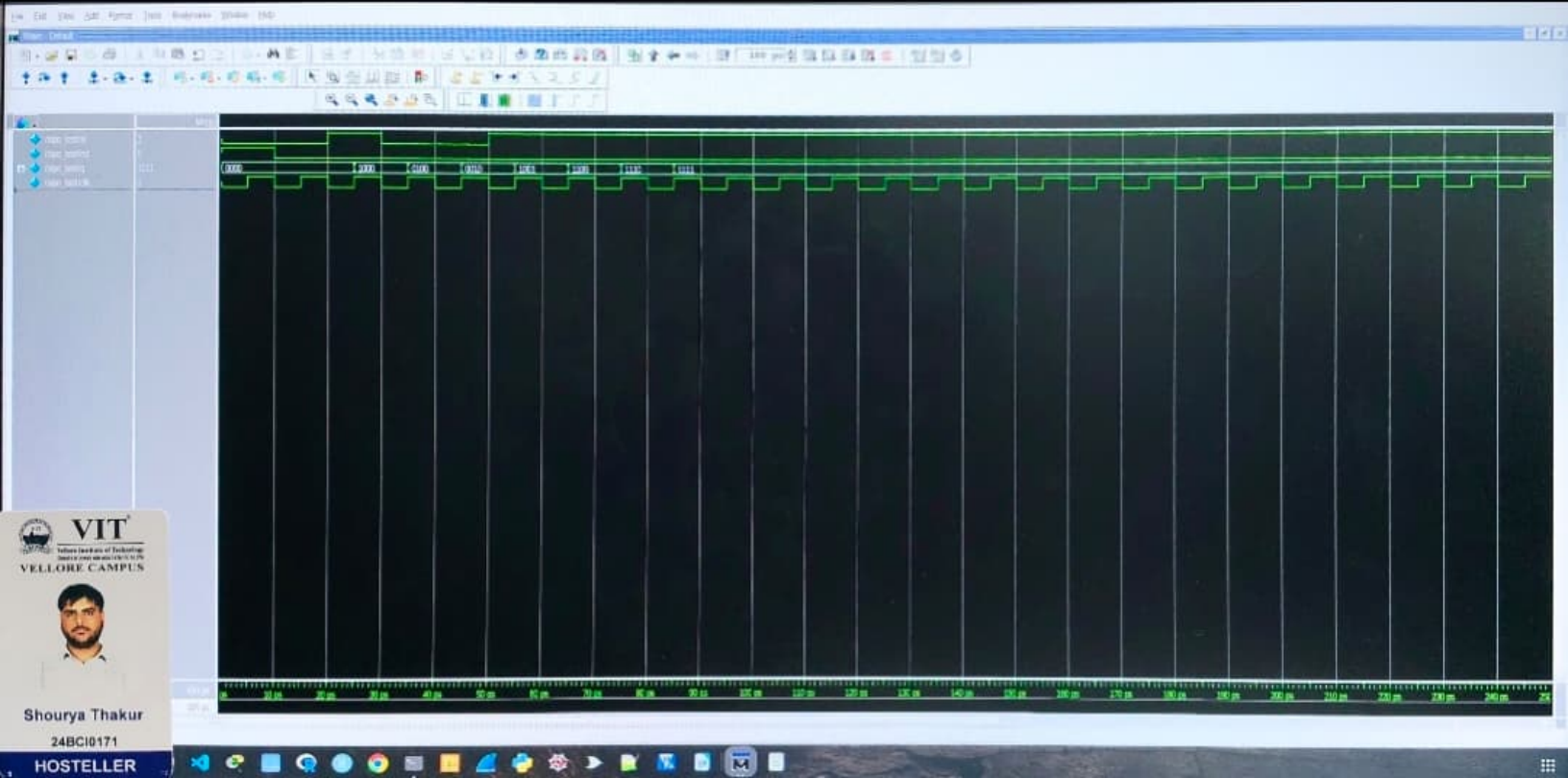
VIT
Vellore Institute of Technology
VELLORE CAMPUS



Shourya Thakur

24BCI0171

HOSTELLER



VIT
Vellore Institute of Technology
VELLORE CAMPUS



Shourya Thakur
24BCI0171
HOSTELLER

Inference:

The SISO and SIPO shift registers were successfully designed and simulated using ModelSim. The output waveforms verified that data in SISO was shifted serially through each flip flop, while in SIPO, the data entered serially was available in parallel at the outputs after the required clock pulses.

Hence, the simulation results confirm the theoretical operation of both SISO and SIPO shift registers.

Aim :- To design, simulate, and verify the working of Up-down Counter, Mod-N Counter, and Johnson Counter using ModelSim.

Tools :- ModelSim

Theory :-

A counter is a sequential circuit that counts the number of clock pulses applied at its input. Counters are widely used in digital systems for timing, frequency division, event counting, and sequencing operations. They are typically built using flip-flops, and their outputs change in a defined binary sequence based on the type of counter.

(a) Up-down Counter

- An up-down counter can count both incrementally (up) and decrementally (down) depending on a control signal.
- When the control input is HIGH, the counter counts up (binary increment).
- When LOW, it counts downward (binary decrement).
- It is useful in digital clocks, timers, and digital devices requiring reversible counting.

truth table (3-bit up-down)

Clock Pulse	Mode (Control)	Count (Binary)	Count (Decimal)
1	up (1)	001	1
2	up (1)	010	2
3	up (1)	011	3
4	up (1)	100	4
5	down (0)	011	3
6	down (0)	010	2
7	down (0)	001	1
8	down (0)	000	0

up down Counter

```
module up-down (Q, up-down, clk, rst);  
output [3:0] Q;  
input up-down, clk, rst;  
reg [3:0] Q;  
always @ (posedge clk or posedge rst)
```

begin

if (rst)

Q <= 4'b0;

else if (up-down)

Q <= Q + 1;

else

Q <= Q - 1;

end

endmodule

module up-down-test

win [3:0] Q;

reg up-down, clk, rst;

up-down <= 1 (Q, up-down, clk, rst);

always

5 clk = ~clk;

initial

begin

clk = 0; up-down = 1; rst = 1;

10 rst = 0;

100 up-down = 0;

200 \$ stop

end

endmodule

o/p
s. by
T. 10/20
24 Dec 2017

(b) MOD-N Counter

- A MOD-N counter goes through N distinct states before resetting to 0.
- The "MOD" value defines the total number of unique states (counts).
- eg, a MOD-8 counter counts from 0 to 7 and then resets to 0 the next clock pulse.
- The number of flip-flops required is given by $N \leq 2^n$, where n is the number of flip-flops.

Truth table for MOD-8 counter

Clock Pulse	Output (Binary)	Output (decimal)
1	0 0 0	0
2	0 0 1	1
3	0 1 0	2
4	0 1 1	3
5	1 0 0	4
6	1 0 1	5
7	1 1 0	6
8	1 1 1	7
9	0 0 0	0 (Reset)

Mod N counter (N=10)

module modN_counter (Q, clk, rst);

output [3:0] Q;

input clk, rst;

reg [3:0] Q;

always @ (posedge clk or posedge rst)

begin

if (rst)

Q <= 4'b0;

else if (Q == 9)

Q <= 4'b0;

~~Q <= Q + 1;~~ ~~1010~~ ~~1010~~ ~~1010~~

end

end module

module modN_counter_test;

wire [3:0] Q;

reg clk, rst;

modN_counter m1(Q, clk, rst);

always

5 clk = ~clk

initial

begin

clk = 0; rst = 1;

10 rst = 0;

200 \$ stop;

end

end module

(c) Johnson Counter

- A Johnson counter, also known as a Twisted Ring Counter, is a type of shift register counter.
- The complemented output of the last flip-flop is fed back as input to the first flip-flop.
- It cycles through $2N$ unique states for N flip-flops, unlike a normal ring counter which has only N states.
- Commonly used for sequence generation, timing circuits, and pattern detection.

Truth table for 4-bit Johnson Counter

Clock Pulse	Q ₃	Q ₂	Q ₁	Q ₀	Sequence (Binary)
1	0	0	0	1	0001
2	0	0	1	1	0011
3	0	1	1	1	0111
4	1	1	1	1	1111
5	1	1	1	0	1110
6	1	1	0	0	1100
7	1	0	0	0	1000
8	0	0	0	0	0000

Johnson Counter

module johnson - counter (q, clk, rst);

output [3:0] q;

input clk, rst;

reg [3:0] q;

always @ (posedge clk or posedge rst)

begin

if (rst)

q <= 4'b0000;

else

q <= {~q[0], q[3:1]};

end

endmodule

module johnson - counter - test;

width [3:0] q;

reg clk, rst;

johnson - counter j1 (q, clk, rst);

always

5 clk = ~clk;

initial

begin

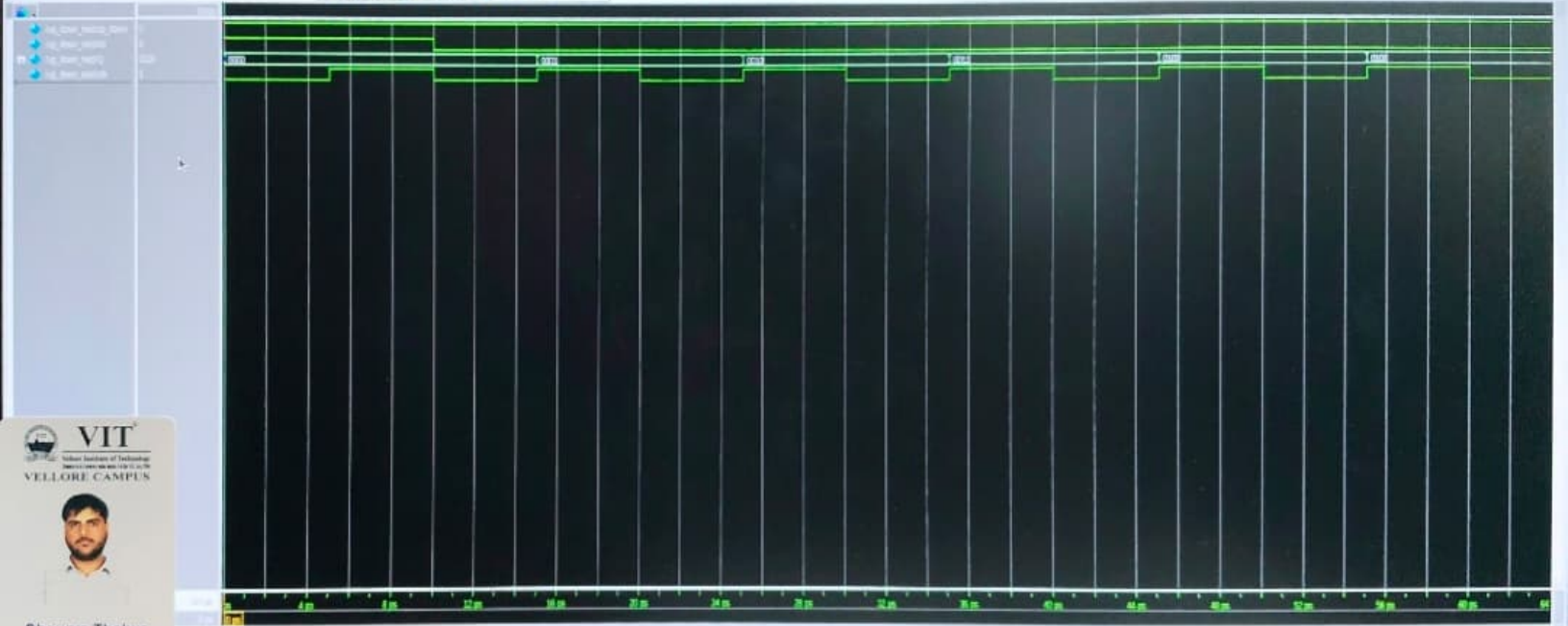
clk = 0; ~~rst = 1;~~ ^{o/p} 10h 24 B 010101

10 rst = 0;

200 \$stop;

end

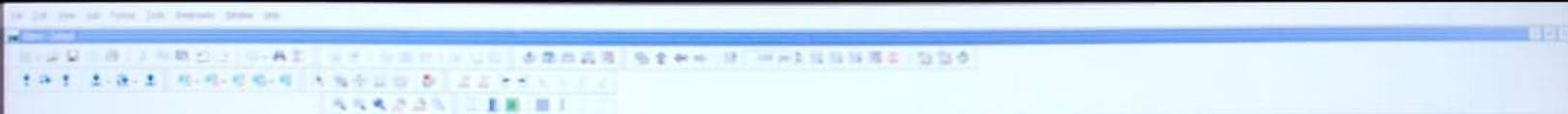
endmodule



 **VIT**
Vellore Institute of Technology
VELLORE CAMPUS



Shourya Thakur
24BCI0171
HOSTELLER



VIT
Vellore Institute of Technology
VELLORE CAMPUS

Shourya Thakur
24BC10171
HOSTELLER



 **VIT**
Vellore Institute of Technology
VELLORE CAMPUS



Shourya Thakur
24BCI0171
HOSTELLER

Conclusion :

The Up-down counter, MOD-N counter, and Johnson Counter were successfully designed, simulated, and verified using ModelSim.

The simulated waveform matched the theoretical truth tables, confirming proper operation.

↳ The Up-down counter demonstrated reversible counting.

↳ The MOD-N counter correctly cycled through N states and reset after reaching its limit.

↳ The Johnson Counter produced a unique sequence of $2N$ states with feedback connection.

Hence, all counters were verified to work as per theoretical design.

	1	0	0	0	0	1
1100	1	1	0	0	0	2
1010	1	1	1	0	0	3
1001	1	1	1	1	0	4
0111	0	1	1	1	1	5
0011	0	0	1	1	1	6
0001	0	0	0	1	1	7
0000	0	0	0	0	1	8